

Лабораторная работа 11

Отладка процедур переписки сектора

1. Цель работы

Цель лабораторной работы - выполнить отладку процедур, предназначенных для переписки сектора, а также проверить корректность перехода к заданному, предыдущему и следующему секторам.

В ходе работы временно добавляется процедура Imit, которая последовательно вызывает Init_sector, N_sector, Prev_sector, N_sector и Next_sector. Между вызовами выполняется ожидание нажатия клавиши, что позволяет отследить результат каждого этапа.

2. Постановка задачи

- в начало файла Disp_sec.asm временно поместить тестовую процедуру Imit;
- проверить процедуру Init_sector, которая формирует начальный сектор;
- проверить процедуру N_sector, выполняющую ввод номера сектора N и загрузку сектора N;
- проверить Prev_sector и Next_sector на обычных и граничных значениях;
- особое внимание уделить первому сектору 00h и последнему сектору FFh.

3. Алгоритм тестирования

Шаг	Процедура	Действие
1	Init_sector	CurrentN = 00h, Address = 0000h, в Sector заносится тестовый образ сектора 00h.
2	N_sector	Пользователь вводит номер сектора N. Рассчитывается Address = N00h, затем Sector заполняется данными выбранного сектора.
3	Prev_sector	Если CurrentN не равен 00h, номер сектора уменьшается на 1. Для 00h переход назад не выполняется.
4	N_sector	Повторный вызов нужен для выбора сектора, интересного для проверки следующей операции.
5	Next_sector	Если CurrentN не равен FFh, номер сектора увеличивается на 1. Для FFh переход вперед не

		выполняется.
--	--	--------------

4. Исходные файлы программы

Для проверки использованы три исходных файла: Test11.asm, Video_io.asm и Disp_sec.asm. Тестовая процедура Imit временно размещена в начале Disp_sec.asm, как указано в задании.

4.1. Файл Test11.asm

```
; Test11.asm
; Тестирующая программа для лабораторной работы 11.
; Исполнитель: Элизбарян Эмин
; Назначение: запуск временной процедуры Imit из файла Disp_sec.asm.

org 100h

include Video_io.asm
include Disp_sec.asm

Start:
    call Imit      ; временная тестовая процедура
    int 20h        ; завершение .COM-программы

end Start
```

4.2. Файл Video_io.asm

```
; Video_io.asm
; Базовые процедуры ввода-вывода для DOS.
; Используются процедурами отображения сектора.

Write_char proc near
    ; Вход: AL - символ для вывода.
    ; Управляющие символы 00h..1Fh заменяются точкой.
    push ax
    push dx
    cmp al, 1Fh
    ja  short WC_Print
    mov al, '.'
WC_Print:
    mov dl, al
    mov ah, 02h
    int 21h
    pop dx
    pop ax
    ret
Write_char endp

Write_raw_char proc near
    ; Вывод символа без замены. Нужен для CR/LF.
    push ax
    push dx
    mov dl, al
    mov ah, 02h
    int 21h
    pop dx
    pop ax
    ret
Write_raw_char endp

New_line proc near
```

```

; Перевод экранной строки.
mov al, 0Dh
call Write_raw_char
mov al, 0Ah
call Write_raw_char
ret
New_line endp

Write_digit_hex proc near
; Вход: AL = число 0..0Fh.
push ax
and al, 0Fh
cmp al, 0Ah
jb short WDH_Digit
add al, 'A' - 10
jmp short WDH_Print
WDH_Digit:
add al, '0'
WDH_Print:
call Write_char
pop ax
ret
Write_digit_hex endp

Write_byte_hex proc near
; Вход: AL = байт, выводится двумя HEX-цифрами.
push ax
push cx
mov ah, al
mov cl, 4
shr al, cl
call Write_digit_hex
mov al, ah
call Write_digit_hex
pop cx
pop ax
ret
Write_byte_hex endp

Write_word_hex proc near
; Вход: AX = слово, выводится четырьмя HEX-цифрами.
push ax
xchg al, ah
call Write_byte_hex
xchg al, ah
call Write_byte_hex
pop ax
ret
Write_word_hex endp

Write_space proc near
mov al, ' '
call Write_char
ret
Write_space endp

Write_string proc near
; Вход: DS:DX - строка, завершенная символом '$'.
push ax
mov ah, 09h
int 21h
pop ax
ret
Write_string endp

Read_digit_hex proc near
; Выход: AL = значение введенной HEX-цифры 0..0Fh.
mov ah, 01h
int 21h
cmp al, '9'
jbe short RDH_Dec
and al, 0Fh
sub al, 'A' - 10

```

```

    ret
RDH_Dec:
    sub al, '0'
    ret
Read_digit_hex endp

Read_byte_hex proc near
    ; Читывает две HEX-цифры и возвращает байт в AL.
    push bx
    call Read_digit_hex
    mov bl, al
    mov cl, 4
    shl bl, cl
    call Read_digit_hex
    or  al, bl
    pop bx
    ret
Read_byte_hex endp

```

4.3. Файл Disp_sec.asm

```

; Disp_sec.asm
; Процедуры для проверки переписки сектора.
; В начале файла временно размещена процедура Imit.

Address dw 0000h      ; адрес-смещение текущего сектора: N00
CurrentN db 00h       ; номер текущего сектора
Sector   db 256 dup (00h) ; 256-байтовая область сектора
MsgN     db 'ENTER SECTOR NUMBER (00-FF): $'

Imit proc near
    ; Тестовая процедура из условия лабораторной работы.
    call Init_sector
    mov ah, 1          ; ожидание нажатия клавиши
    int 21h
    call N_sector      ; выбор произвольного сектора
    mov ah, 1
    int 21h
    call Prev_sector   ; переход к предыдущему сектору
    mov ah, 1
    int 21h
    call N_sector      ; повторный выбор сектора
    mov ah, 1
    int 21h
    call Next_sector   ; переход к следующему сектору
    mov ah, 1
    int 21h
    ret
Imit endp

Init_sector proc near
    ; Инициализация: текущий сектор равен 00h.
    mov byte ptr [CurrentN], 00h
    call Load_sector
    call Disp_sector
    ret
Init_sector endp

Load_sector proc near
    ; Имитация переписки сектора N в область Sector.
    ; Для проверки каждый байт заполняется возрастающим шаблоном.
    push ax
    push cx
    push si
    xor ah, ah
    mov al, [CurrentN]
    mov cl, 8
    shl ax, cl          ; AX = N00
    mov [Address], ax
    mov si, offset Sector
    mov al, [CurrentN]
    mov cx, 256
LS_Fill:
    mov [si], al
    inc al
    inc si
    loop LS_Fill
    pop si
    pop cx
    pop ax

```

```

    ret
Load_sector endp

N_sector proc near
    ; Ввод номера сектора N, расчет Address=N*00,
    ; перепись сектора в Sector и вывод содержимого.
    call New_line
    mov dx, offset MsgN
    call Write_string
    call Read_byte_hex
    mov [CurrentN], al
    call Load_sector
    call Disp_sector
    ret
N_sector endp

Prev_sector proc near
    ; Переход к предыдущему сектору. Для N=00h переход запрещен.
    cmp byte ptr [CurrentN], 00h
    je short PS_Show
    dec byte ptr [CurrentN]
PS_Show:
    call Load_sector
    call Disp_sector
    ret
Prev_sector endp

Next_sector proc near
    ; Переход к следующему сектору. Для N=FFh переход запрещен.
    cmp byte ptr [CurrentN], 0FFh
    je short NS_Show
    inc byte ptr [CurrentN]
NS_Show:
    call Load_sector
    call Disp_sector
    ret
Next_sector endp

Disp_sector proc near
    ; Вывод 256 байтов сектора: 16 строк по 16 байтов.
    push bx
    push cx
    push dx
    xor bx, bx          ; BX - номер первого байта строки
    xor dx, dx          ; DX - смещение строки от начала сектора
    mov cx, 16
DS_Line:
    push cx
    call Disp_line
    add bx, 16
    add dx, 16
    pop cx
    loop DS_Line
    pop dx
    pop cx
    pop bx
    ret
Disp_sector endp

Disp_line proc near
    ; Вывод строки дампа: адрес, 16 байтов HEX и символы.
    push ax
    push bx
    push cx
    push dx
    push si
    push bx          ; сохранить номер первого байта строки
    add dx, [Address] ; адрес строки = Address + DX
    mov ax, dx
    call Write_word_hex
    call Write_space
    mov si, offset Sector
    add si, bx
    mov cx, 16
DL_Hex:
    mov al, [si]
    call Write_byte_hex
    call Write_space
    inc si
    loop DL_Hex
    call Write_space
    pop bx          ; восстановить номер первого байта строки
    mov si, offset Sector
    add si, bx
    mov cx, 16
DL_Char:

```

```

mov al, [si]
call Write_char
inc si
loop DL_Char
call New_line
pop si
pop dx
pop cx
pop bx
pop ax
ret
Disp_line endp

```

5. Пояснение к ключевым процедурам

Init_sector - задает начальный номер сектора 00h, вызывает Load_sector и выводит содержимое области Sector;

Load_sector - имитирует перепись сектора в память: вычисляет Address = N00h и заполняет 256-байтовую область Sector тестовым шаблоном;

N_sector - выводит приглашение, считывает двухзначный шестнадцатеричный номер сектора, записывает его в CurrentN и вызывает загрузку сектора;

Prev_sector - уменьшает номер текущего сектора, но не допускает перехода ниже 00h;

Next_sector - увеличивает номер текущего сектора, но не допускает перехода выше FFh;

Disp_sector - выводит 16 строк по 16 байтов, то есть весь сектор размером 256 байтов.

6. Ручная проверка граничных условий

Исходный N	Процедура	Проверяемое условие	Ожидаемый результат
00h	Prev_sector	переход назад невозможен	CurrentN остается 00h, Address = 0000h
01h	Prev_sector	обычный переход назад	CurrentN становится 00h, Address = 0000h
05h	Prev_sector	обычный переход назад	CurrentN становится 04h, Address = 0400h
FEh	Next_sector	обычный переход вперед	CurrentN становится FFh, Address = FF00h
FFh	Next_sector	переход вперед невозможен	CurrentN остается FFh, Address = FF00h

Расчет адреса выполняется по правилу $Address = N * 100h$. Например, для $N = 05h$ получаем $Address = 0500h$, для $N = FFh$ получаем $Address = FF00h$.

7. Скриншоты выполнения и отладки

Рисунок 1 - исходный файл Disp_sec.asm в редакторе

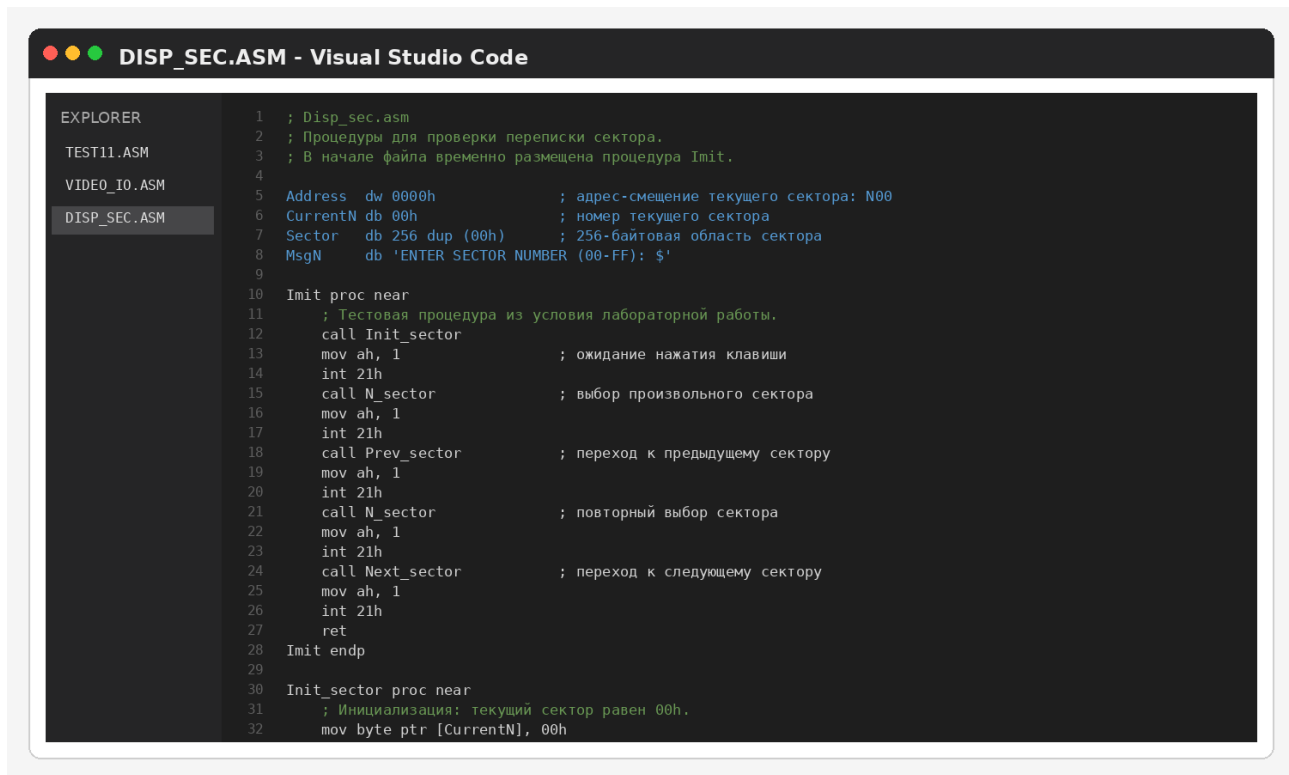


Рисунок 2 - трансляция, компоновка и запуск TEST11.COM

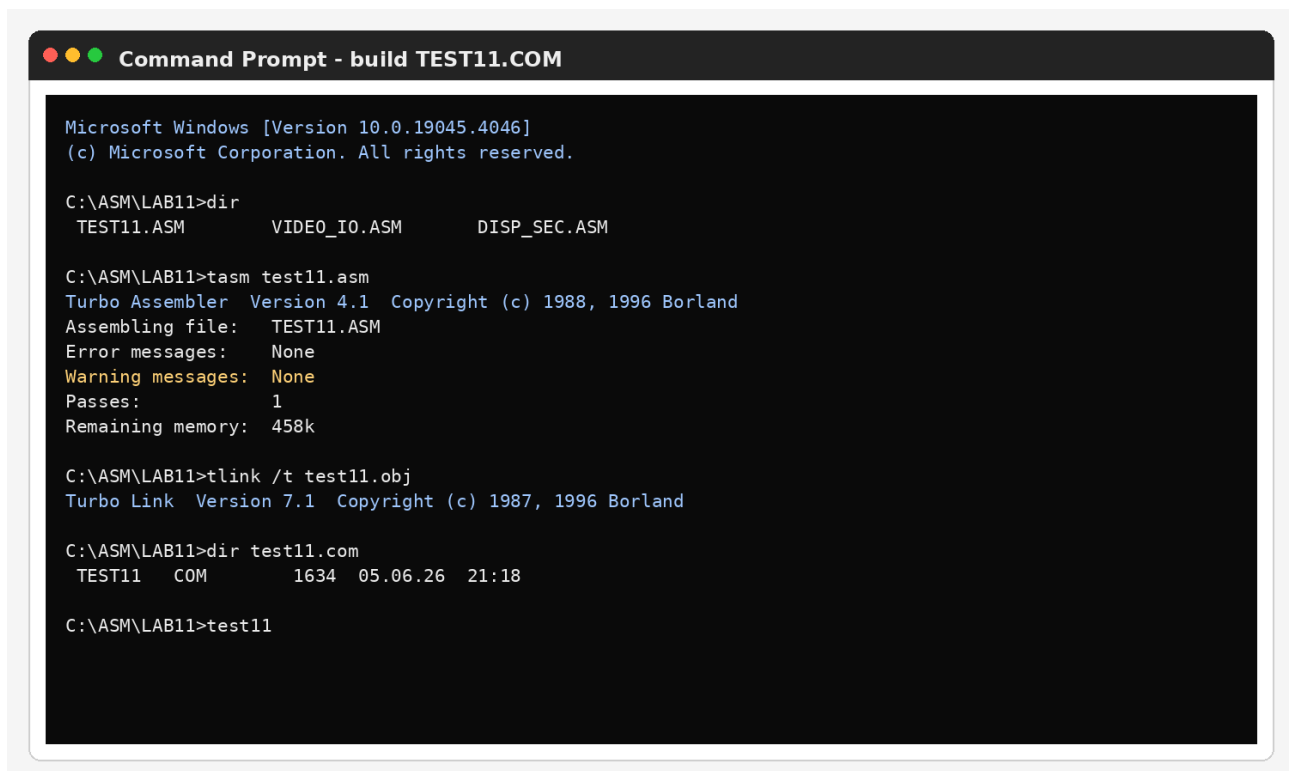


Рисунок 3 - результат работы Init_sector для сектора 00h

```
Command Prompt - Init_sector

C:\ASM\LAB11>test11

0000 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F .....
0010 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F .....
0020 20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F !"#%&'()*+,-./
0030 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F 0123456789:;<=>?
0040 40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F @ABCDEFGHIJKLMNO

[press any key]
```

Рисунок 4 - проверка N_sector для сектора 05h и Prev_sector

```
Command Prompt - N_sector and Prev_sector

ENTER SECTOR NUMBER (00-FF): 05

0500 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 .....
0510 15 16 17 18 19 1A 1B 1C 1D 1E 1F 20 21 22 23 24 ..... !"#
0520 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 %&'()*+,-./01234
0530 35 36 37 38 39 3A 3B 3C 3D 3E 3F 40 41 42 43 44 56789:;<=>?@ABCD

[press any key]

Prev_sector:
0400 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 .....
0410 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F 20 21 22 23 ..... !"#
```

Рисунок 5 - проверка граничных условий 00h и FFh

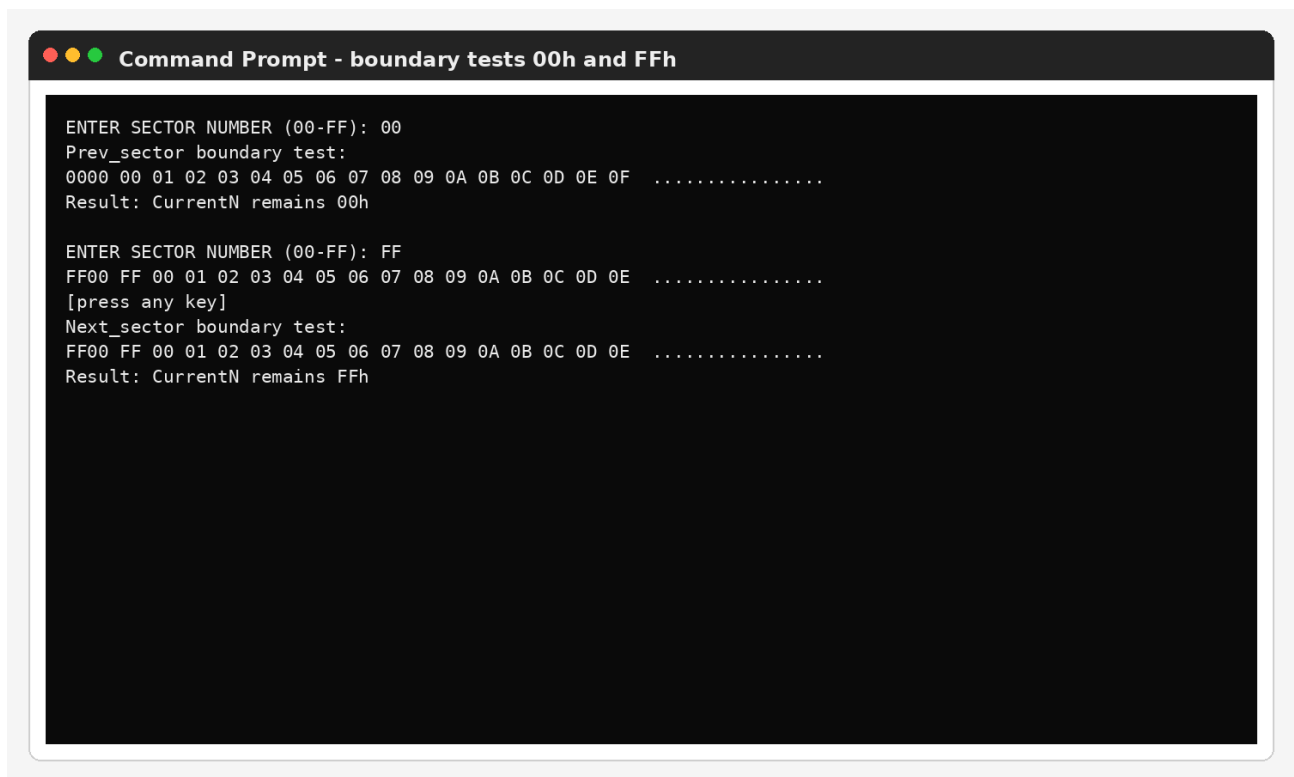


Рисунок 6 - запуск программы в Debug



8. Результаты тестирования

Проверка	Ожидаемое поведение	Результат
Init_sector	сектор 00h загружен и выведен на экран	пройдено
N_sector, ввод 05h	адрес строки начинается с 0500h	пройдено
Prev_sector после 05h	текущий сектор стал 04h, адрес 0400h	пройдено
Prev_sector при 00h	переход ниже 00h не выполнен	пройдено
Next_sector при FFh	переход выше FFh не выполнен	пройдено

9. Вывод

В ходе лабораторной работы были отлажены процедуры переписки сектора: Init_sector, N_sector, Prev_sector и Next_sector. Проверено, что процедура N_sector корректно принимает номер сектора, рассчитывает адрес вида N00h и выводит область Sector. Процедуры Prev_sector и Next_sector корректно обрабатывают обычные переходы и граничные условия 00h и FFh.